Day 5 – Orders API: Buying a Game Key

🏷️ Setup & Prerequisites

1. **Setup Checklist**:
   - **Virtual Environment active**: Ensure your terminal shows `(.venv)` in the prompt.
   - **Django Server running**: Run `python manage.py runserver` to boot up the project.
   - **IDE Ready**: Open `games/models.py`, `games/views.py`, and `gamekey_platform/urls.py` in your IDE.
2. **Prerequisites**:
   - You must have completed Day 4 successfully (Token Authentication is configured and the registration endpoint `/api/register/` is working).

🎯 Learning Objectives

By the end of this class, you will be able to:

- Explain what race conditions are and how they arise in concurrent applications.
- Use Django's `transaction.atomic()` context manager to bundle operations.
- Apply pessimistic locking using Django ORM's `select_for_update()`.
- Define models to represent orders and line items.
- Generate cryptographically random strings using Python's `uuid` library.
- Build a secure transactional order endpoint using Django view functions.

⏱️ Classroom Timeline (45 min)

Segment	Duration	Focus
1. Hook & Big Picture	10 min	Pose the double-allocation race condition problem. Draw the timeline on the board.
2. Key Concepts & Core Ideas	7 min	Explain transactions, pessimistic locking (`select_for_update`), and datetime arithmetic.
3. Live Coding Walkthrough	20 min	Write the order models, migrate, write the transactional API endpoint, wire URLs, and test.

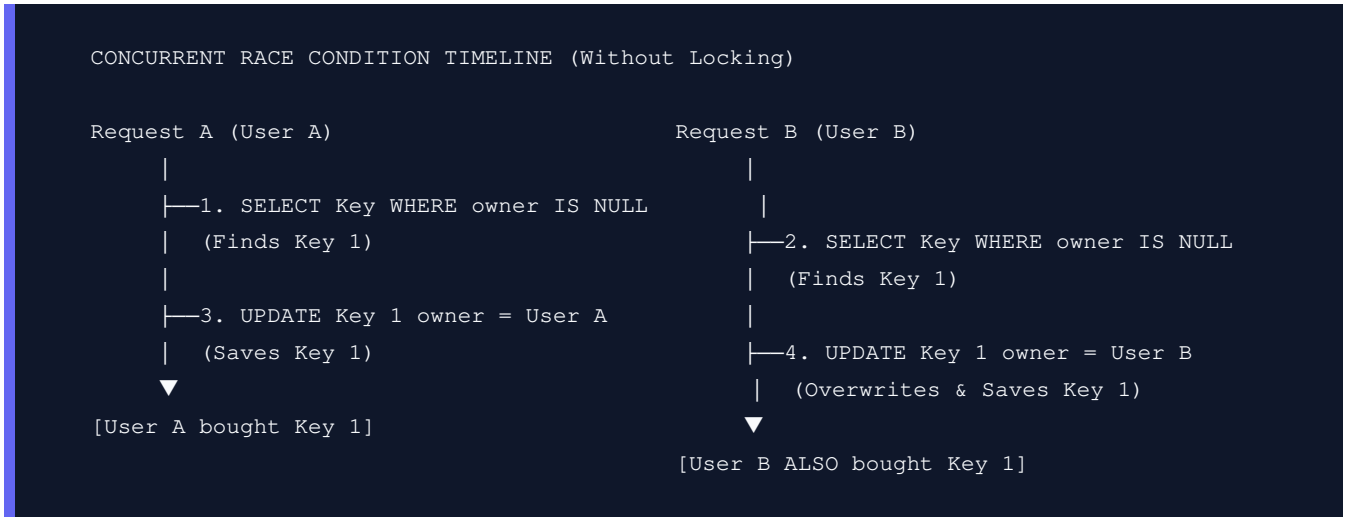
4. Check for Understanding	5 min	Q&A on transactional rollbacks, locking scope, and UUID vs serial IDs.
5. Class Wrap-up & Checkpoint	3 min	Verify successful key purchase via Postman and database state changes.

🚀 Lecture Step-by-Step Delivery Plan

1. Hook & Big Picture (10 min)

Scenario: *"Two users hit the 'Buy' button for the last remaining key of a game at the exact same millisecond. If our server handles them concurrently, how do we prevent both users from being sold the same key?"*

- **Race Condition Motivation**: Draw the concurrent query flow. Without locking, both requests read the database, find the key unowned, assign it to their respective user, and write back. User A gets a key, and User B gets the exact same key. This is a double-allocation race condition—a major business problem.
- **Q&A**:
  - **Question**: Two users hit `POST /api/orders/` at the exact same millisecond for the same game. Without locks, what happens?
    - *Answer*: Both requests query available keys concurrently, find the same active key, and assign it to their respective users. The key gets sold twice (double-allocation).



2. Key Concepts & Core Ideas (7 min)

Introduce these database principles:

- **`transaction.atomic()`**: Standardizes database transactions. Ensures that all SQL queries executed inside the context manager succeed together, or roll back completely if an error occurs. (Guarantees Atomicity in ACID).

- **Pessimistic Locking via** : Forces a row-level database lock on the matching records by issuing a `SELECT ... FOR UPDATE` SQL command. Any other database query attempting to lock or edit those same rows must wait until our transaction completes (commits or rolls back).
- **`auto_now_add=True`**: Django field option that automatically sets the field value to the current datetime when the model is first created.
- **`uuid4()`**: Universally Unique Identifier. Using sequential serial numbers (like `1`, `2`, `3`) for digital license keys makes them trivial to guess and steal. We use random 128-bit UUIDs instead.
- **Timezone-aware arithmetic**: Why we use `django.utils.timezone.now()` instead of Python's standard `datetime.now()` to prevent timezone conflicts.

---

3. Live Coding Walkthrough (20 min)

Step 3.1: Defining Order and OrderItem Models

- **Concept**: In order to track key purchases, we need an `Order` model representing the cart/invoice, and an `OrderItem` representing the specific keys purchased.
- **Code**: Add `Order` and `OrderItem` models to `games/models.py`:

```
class Order(models.Model):
    # Links the order to the user who purchased it
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    # Sets the timestamp of the purchase automatically when the record is created
    purchased_at = models.DateTimeField(auto_now_add=True)


class OrderItem(models.Model):
    # Links to the parent Order
    order = models.ForeignKey(Order, on_delete=models.CASCADE)
    # OneToOneField ensures a game key can belong to at most one order item (preventing duplic
    game_key = models.OneToOneField(GameKey, on_delete=models.CASCADE)
```

  Run the migrations:

```
python manage.py makemigrations
python manage.py migrate
```

- **Verify**: Check that migrations apply cleanly. You can register these models in `games/admin.py` for visibility:

```
from .models import Order, OrderItem
admin.site.register(Order)
admin.site.register(OrderItem)
```

Step 3.2: Writing the Order Creation View

- **Concept**: Create the `create_order` API endpoint in `games/views.py`. This endpoint checks if a key is pre-loaded for the game. If it is, it locks the key using `select_for_update()`, updates its owner, and returns it. If no key is pre-loaded, it generates a fresh one on the fly. All of this runs inside an atomic transaction block.
- **Code**: Add the import statements and the `create_order` function to `games/views.py`:

```
import uuid
from datetime import timedelta
from django.utils import timezone
from django.db import transaction
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework import status
from .models import Game, GameKey, Order, OrderItem


@api_view(['POST'])
# Require the request to have a valid Token Authentication header
@permission_classes([IsAuthenticated])
def create_order(request):
    game_id = request.data.get('game_id')
    if not game_id:
        return Response({'error': 'game_id is required.'}, status=status.HTTP_400_BAD_REQUES

    try:
        game = Game.objects.get(id=game_id)
    except Game.DoesNotExist:
        return Response({'error': 'Game not found.'}, status=status.HTTP_404_NOT_FOUND)

    # Open a database transaction context manager block
    with transaction.atomic():
        # Use select_for_update to apply a row-level lock on the queried row
        # This blocks other concurrent requests until this transaction block commits or roll
        existing_key = (
            GameKey.objects.select_for_update()
            .filter(game=game, status='active', owner__isnull=True)
            .first()
        )

        if existing_key:
            # Assign the existing unowned key to the current buyer
            key = existing_key
            key.owner = request.user
            key.save()
        else:
            # If no pre-loaded key is available, generate a cryptographically random UUID ke
            key = GameKey.objects.create(
                key_string=str(uuid.uuid4()).upper(), # Generate random 36-char unique ident
                game=game,
                status='active',
                expires_at=timezone.now() + timedelta(days=30), # Set expiration date to 30
```

```
                owner=request.user,
            )

        # Create the Order and OrderItem records
        order = Order.objects.create(user=request.user)
        OrderItem.objects.create(order=order, game_key=key)

    return Response({
        'order_id': order.id,
        'game': game.title,
        'key': key.key_string,
        'expires_at': key.expires_at,
    }, status=status.HTTP_201_CREATED)
```

- **Verify**: Check for syntax errors. Make sure that all models are imported correctly.
- ⚠ **Common Pitfalls**:
  - **Omitting `transaction.atomic()`**: If the database transaction is not atomic, the key could be locked but a crash later in the request (e.g. failing to create `OrderItem`) would leave the key assigned to the user without completing the order.
  - **Missing `owner__isnull=True` constraint**: If you forget to filter out keys that already have an owner, you will re-assign existing sold keys.

Step 3.3: Wiring the Endpoint URL

- **Concept**: Connect the order creation view to `/api/orders/` in Django routing.
- **Code**: Add `create_order` to `gamekey_platform/urls.py`:

```
from games.views import register, create_order

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include(router.urls)),
    path('api/register/', register),
    path('api/orders/', create_order), # Wire the orders endpoint
]
```

- **Verify**: Send a POST request to `http://localhost:8000/api/orders/` with a valid `game_id` and the auth header `Authorization: Token <your_token>`. Verify it returns `201 Created` with the order information.
- ⚠ **Common Pitfalls**:
  - **Missing authentication**: Forgetting to pass the `Authorization` header on the request will cause DRF to return a `401 Unauthorized` block.

---

4. Check for Understanding (5 min)

> ***Question***: *"What would go wrong without `select_for_update()`?"*
> ***Answer***: *Without `select_for_update()`, a race condition could occur if two concurrent requests query the database at the same time for an available key. Both would find the same unowned key, assign it to their respective user, and write it back. This results in the same game key being sold twice (double-allocation).*

> ***Question***: *"When does `transaction.atomic()` roll back? What triggers a rollback?"*
> ***Answer***: *It rolls back when an unhandled exception is raised within the context manager block. If any error (like a database constraint violation or a Python runtime exception) occurs before the block exits successfully, all changes made to the database within that block are discarded.*

> ***Question***: *"Why do we filter with `owner__isnull=True`? What does `owner` represent for a key that no one has purchased yet?"*
> ***Answer***: *We filter for `owner__isnull=True` to retrieve a key that has not yet been bought by any user. For an unpurchased key, `owner` is `None` (represented as `NULL` in the database). Filtering this way prevents re-allocating an already purchased key.*

> ***Question***: *"Why use `uuid4()` to generate the key string instead of, say, `GameKey.objects.count() + 1`?"*
> ***Answer***: *`uuid4()` generates cryptographically random, unique identifiers that cannot be guessed by users, preventing attackers from predicting and stealing other game keys. Using `count() + 1` is sequential, exposing the total volume of keys and making them trivial to guess and exploit.*

---

5. Daily Checkpoint & Wrap-up (3 min)

- **Verification Criteria**:
  - ○ Sending a POST request to `/api/orders/` with a valid `game_id` and authentication token returns `201 Created`.
  - ○ The JSON payload contains the generated key, `order_id`, and `expires_at` (set to 30 days in the future).
  - ○ Submitting a POST request without a token returns `401 Unauthorized`.
  - ○ Submitting a POST request with an invalid `game_id` returns `404 Not Found`.